

Administración de Proyectos

BUGLOG

Parcial 2

Comisión:

ACN5BV

Docente:

Sergio Medina

Alumnos:

Lukas Galarza

Mariano Williams

Anthony Salazar

BugLog es una aplicación web fullstack orientada a equipos de QA que trabajan en la industria de los videojuegos. Su propósito es centralizar el proceso de reporte y gestión de bugs, reemplazando métodos informales como planillas, correos o mensajes de texto por una herramienta estructurada, segura y accesible. El sistema contempla dos perfiles de usuario con acceso diferenciado: el *Tester*, que puede reportar bugs y gestionar sus propios registros, y el *Administrador*, que cuenta con control total sobre los reportes y la gestión de usuarios de la plataforma.

El presente documento describe la planificación y gestión del desarrollo de **BugLog**, aplicando conceptos de Administración de Proyectos y metodologías ágiles para organizar el trabajo del equipo y gestionar la evolución de los requerimientos a lo largo del proyecto.

Objetivo general del proyecto – OKR

El objetivo general de BugLog fue definido utilizando **OKR**. A diferencia de SMART, que orienta la redacción de objetivos concretos, acotados y alcanzables dentro de un plazo determinado, OKR nos permite plantear una dirección más ambiciosa y nos da la posibilidad a que el proyecto pueda ser escalable a algo más amplio sin perder la coherencia del mismo. Para un producto de software como **BugLog**, donde los requerimientos pueden crecer, donde nuevas funcionalidades pueden sumarse OKR nos resultó más apropiado.

En términos de escalabilidad, esto marca una diferencia importante. BugLog hoy es una herramienta para equipos pequeños de QA, pero su arquitectura MVC, su sistema de roles y su base de datos relacional están pensados para crecer. Podrían sumarse en el futuro nuevas plataformas soportadas, notificaciones automáticas, reportes estadísticos o funcionalidades más ambiciosas durante su desarrollo, **OKR** permite que ese crecimiento sea parte natural del proyecto.

Lo que se quiere alcanzar con BugLog es que cualquier equipo de QA, sin importar su tamaño, pueda reemplazar sus métodos informales de reporte por una herramienta estructurada, segura y centralizada. Que un tester pueda registrar un bug en segundos con toda la información relevante, que un administrador tenga visibilidad total sobre el estado del testing y que el estudio pueda tomar decisiones de lanzamiento con datos reales en lugar de hilos de mensajes. A donde se quiere llegar es a que BugLog sea una herramienta lo suficientemente robusta y flexible como para adaptarse a distintos estudios, flujos de trabajo y escalas de equipo, sin perder la simplicidad de su uso.

#	Key Result	Validación
KR1	Implementar autenticación JWT con roles diferenciados	Sistema funcional con roles Admin y Tester verificados por `authMiddleware.js` y `verifyTokenAndAdmin`
KR2	Lograr cobertura del 100% de operaciones CRUD en bugs	Endpoints GET, POST, PUT y DELETE en `/api/bugs` con validaciones via `express-validator`
KR3	Proveer panel de administración de usuarios completamente funcional	CRUD completo en `/api/users` restringido a rol admin, con interfaz en `AdminUsers.jsx`
KR4	Asegurar persistencia de datos con base de datos relacional	SQLite con FOREIGN KEY, UNIQUE constraints, CHECK de roles e índices optimizados
KR5	Diseño responsive funcional para desktop, tablet y móvil	CSS modular por componente, grid de 3 columnas responsive, tema oscuro inspirado en Steam

Análisis de interesados — Stakeholders

En el caso de BugLog, los interesados fueron identificados a partir del análisis del contexto de uso de la herramienta: equipos de QA dentro de estudios de videojuegos, de distinto tamaño y estructura organizacional.

Para este proyecto se identificaron los siguientes interesados principales junto a su influencia e impacto:

Estudio de Videojuegos

Empresa o estudio presumiblemente independiente que desarrolla videojuegos y requiere una herramienta centralizada para que su equipo de QA reporte y gestione los errores detectados durante las fases de testing.

Influencia: Alta. Define los requerimientos funcionales del sistema y es el principal financiador del proyecto.

Impacto: Determina qué funcionalidades son prioritarias y con qué criterios se evalúa el éxito del producto. Si el sistema no cubre sus necesidades operativas, optará por herramientas alternativas.

Tester de QA

Profesional de QA que prueba videojuegos en distintas plataformas y necesita una herramienta ágil para registrar bugs durante sus sesiones de testing. Es el usuario más activo del sistema y su experiencia de uso determina directamente la adopción de la herramienta.

Influencia: Media. Su experiencia de uso cotidiano define si la herramienta es adoptada o abandonada.

Impacto: Permite validar la usabilidad del sistema y la efectividad del flujo de reporte de bugs. Un sistema con fricción genera rechazo y retorno a métodos informales.

Líder Técnico / Project Manager del estudio

Responsable técnico que supervisa el proceso de QA, gestiona al equipo de testers y necesita visibilidad sobre el estado general de los bugs para tomar decisiones sobre el lanzamiento de cada título.

Influencia: Alta. Tiene poder de decisión sobre si el sistema reemplaza las herramientas actuales del estudio.

Impacto: Define las reglas de acceso, supervisa la integridad de los datos y valida que el sistema se adapta a la operatoria del equipo.

Jugadores

Personas que adquieren y juegan los videojuegos producidos por el estudio. No interactúan con BugLog directamente, pero son los afectados por la calidad del producto que el sistema contribuye a mejorar.

Influencia: Baja sobre BugLog directamente, pero alta sobre el negocio del estudio.

Impacto: Si el proceso de QA falla y el juego se lanza con errores críticos, las reseñas negativas impactan directamente en las ventas y la reputación del estudio. BugLog contribuye a mitigar este riesgo.

A partir de este análisis se determinó que el Estudio de Videojuegos y el Líder Técnico representan los stakeholders con mayor influencia sobre el proyecto, ya que participan en la definición y validación de los requerimientos funcionales.

Stakeholder	Nivel de Interés	Nivel de influencia
Estudio de videojuegos	Muy alto	Alto
Tester de QA	Alto	Medio
Líder Técnico / Project Manager	Alto	Alto
Jugadores	Medio	Bajo

Metodología Ágil – Scrum

Para el desarrollo de BugLog se adoptó la metodología ágil **Scrum**, gestionada mediante Jira como herramienta de seguimiento. Esta metodología permite organizar el trabajo de manera iterativa e incremental, facilitando la adaptación a cambios en los requerimientos y promoviendo la entrega continua de valor a lo largo del proyecto.

La elección de Scrum se fundamenta en la necesidad de estructurar el trabajo en sprints con objetivos claros y medibles, organizar el backlog mediante historias de usuario con estimaciones en story points y mantener visibilidad del avance del equipo en todo momento. Otros puntos para justificar su elección sería que al ser un equipo pequeño la metodología **Scrum** es adaptable a 3 personas sin necesidad de roles formales rígidos, manteniendo las entregas incrementales; cada sprint permite la evaluación progresiva del proyecto mediante **Jira**.

Para el seguimiento del proyecto se definieron las siguientes ceremonias adaptadas al contexto del equipo:

Sprint Planning: reunión destinada a planificar las historias de usuario a desarrollar en cada sprint, definiendo objetivos, estimaciones y asignaciones.

Daily Meeting: encuentro breve para revisar avances e identificar impedimentos. Dada la naturaleza del proyecto y el tamaño del equipo, estas sincronizaciones se realizan de forma asincrónica mediante mensajes cuando la complejidad de las tareas lo requiere.

Sprint Review: instancia de revisión de las funcionalidades desarrolladas y validación de los resultados respecto de los objetivos definidos para cada sprint.

Sprint Retrospective: reunión orientada a identificar oportunidades de mejora en el proceso de trabajo y definir acciones correctivas para el siguiente sprint.

Análisis de Requerimientos — Historias de Usuario

Los requerimientos de BugLog fueron relevados a partir del análisis del flujo de trabajo de un equipo de QA en la industria del videojuego. Se identificaron las necesidades de dos perfiles de usuario diferenciados — el Tester y el Administrador — y a partir de ellas se formularon las historias de usuario que guiaron el desarrollo del sistema.

Para documentar los requerimientos se utilizó el formato de Historia de Usuario, siguiendo la estructura: *Como [rol], quiero [acción] para [beneficio]*. Cada historia fue asignada a un integrante del equipo y vinculada a su épica correspondiente en Jira.

Sprint 1

ID Jira	Historia de Usuario	Criterio de Aceptación
SCRUM-5	Como tester, quiero iniciar sesión con usuario y contraseña para acceder al sistema.	JWT generado, token en localStorage, redirección a Home tras login exitoso.
SCRUM-6	Como admin, quiero que mi rol sea verificado en cada request para restringir accesos no autorizados.	verifyTokenAndAdmin rechaza con 403 si el rol no es admin.
SCRUM-7	Como sistema, quiero almacenar contraseñas encriptadas para proteger los datos de los usuarios.	Bcrypt aplicado al crear y actualizar usuarios. La contraseña nunca se devuelve en las respuestas.
SCRUM-8	Como tester, quiero ser redirigido al login si no tengo sesión activa.	RequireAuth.jsx redirige a /login sin token válido. RequireAdmin.jsx redirige si el rol no es admin.

Sprint 2

ID Jira	Historia de Usuario	Criterio de Aceptación
SCRUM-9	Como tester, quiero reportar un bug con nombre de juego, plataforma, tipo y gravedad.	POST /api/bugs crea el registro. BugForm.jsx valida los campos en tiempo real. Respuesta 201 con el bug creado.
SCRUM-10	Como tester, quiero ver el historial de todos los bugs reportados.	GET /api/bugs con JOIN a tabla users. Reportes.jsx renderiza una BugCard por cada registro.
SCRUM-11	Como tester, quiero editar mis propios reportes para corregir información incorrecta.	PUT /api/bugs/:id verifica que userId coincida. EditModal.jsx carga los datos previos del reporte.
SCRUM-12	Como admin, quiero editar y eliminar cualquier bug sin restricción de propietario.	El middleware verifica req.user.role === 'admin' para omitir la verificación de userId.
SCRUM-17	Como tester, quiero agregar la portada del juego mediante una URL al reportar un bug.	Campo imageUrl disponible en el formulario. BugCard.jsx muestra la imagen si el campo tiene valor.

Sprint 3

ID Jira	Historia de Usuario	Criterio de Aceptación
SCRUM-13	Como admin, quiero ver una tabla de todos los usuarios registrados del sistema.	GET /api/users disponible solo para admin. AdminUsers.jsx lista id, username, email, rol y fecha de creación.
SCRUM-14	Como admin, quiero crear nuevos usuarios con username, email, contraseña y rol.	POST /api/users con User.create(). Bcrypt aplicado. Validaciones de unicidad de username y email.
SCRUM-15	Como admin, quiero modificar datos de cualquier usuario, incluyendo su rol y contraseña.	PUT /api/users/:id con User.update(). Si se provee nueva contraseña, es hasheada antes de guardarse.
SCRUM-16	Como admin, quiero eliminar usuarios sin poder eliminar mi propia cuenta.	DELETE /api/users/:id con verificación server-side: req.user.id !== id. Confirmación visual en el frontend.

Tablero de Seguimiento – Jira

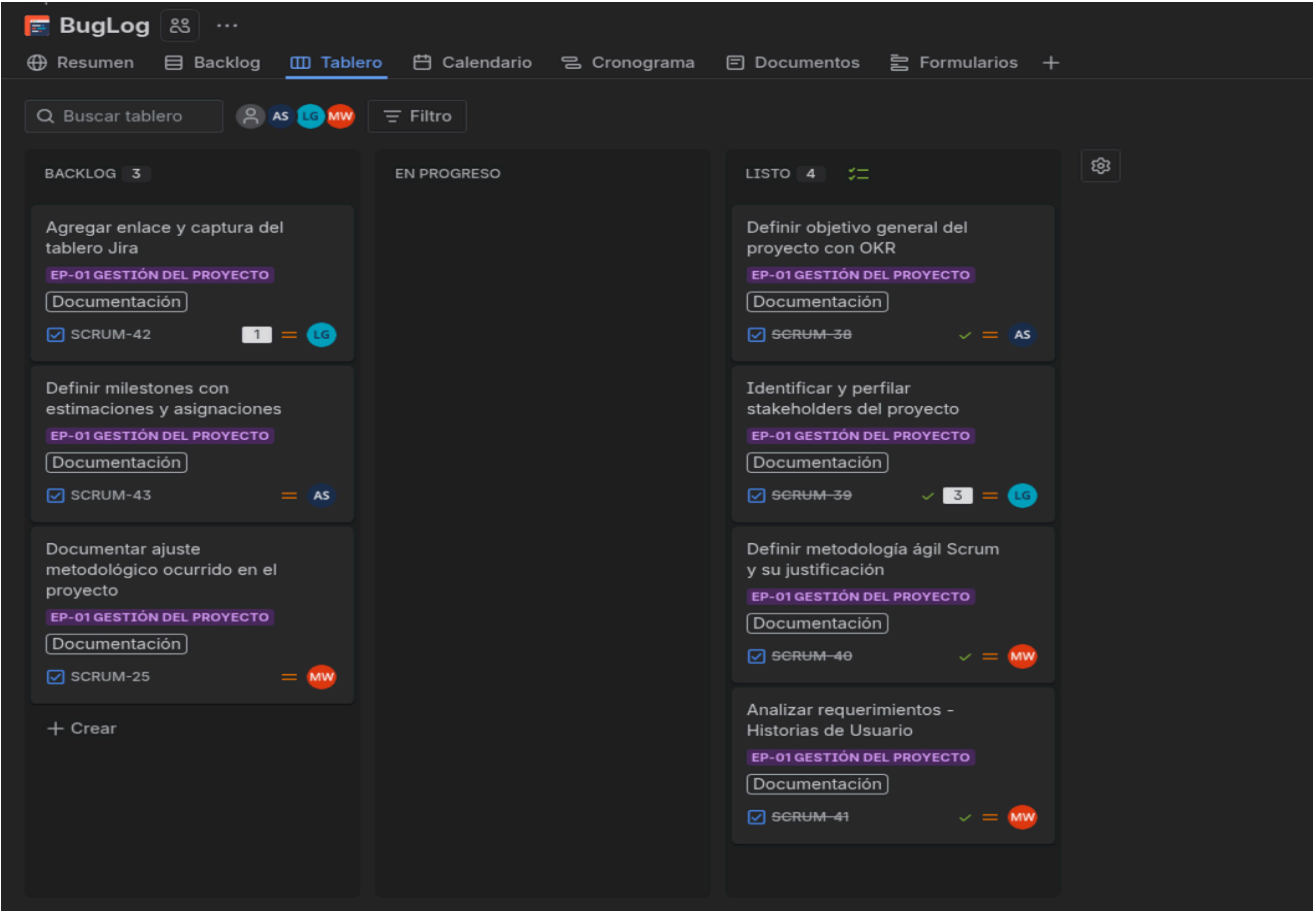
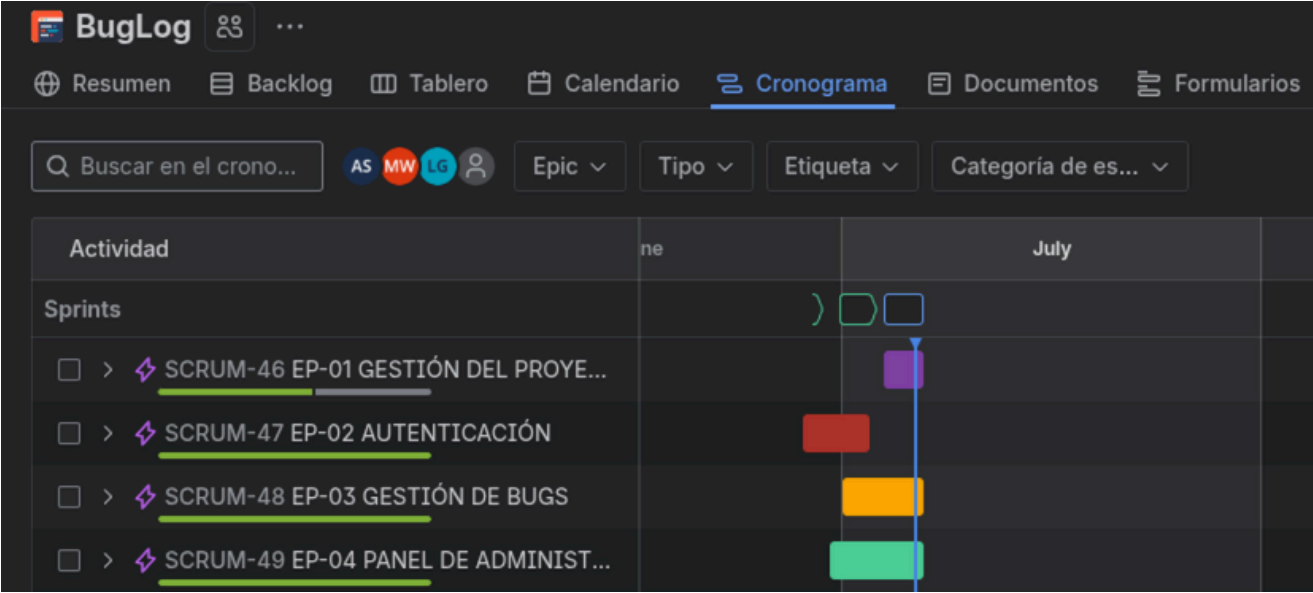
Para el seguimiento del proyecto se configuró un tablero Scrum en Jira, organizado en cuatro columnas que representan el flujo de trabajo del equipo a lo largo de los sprints.

Configuración del Tablero

Columna	Descripción
Backlog	Historias de usuario creadas y pendientes de ser incorporadas al sprint activo.
En Progreso	Ítems activamente en desarrollo durante el sprint en curso.
Listo	Ítems cerrados con funcionalidad verificada y mergeada en la rama principal.
Link al Jira	https://buglog.atlassian.net/jira/software/projects/SCRUM/summary?atlOrigin=eyJpIjoiYjFkZDc5NTdiODkzNGJhNTkyYjIjNzQ2ZmZkOTNhMDMiLCJwIjoiaaiJ9#/rule-list?systemLabelId=project&page=1&pageSize=20&sortKey=name&sortOrder=ASC

Épicas

Épica	Descripción
EP-01 GESTIÓN DEL PROYECTO	Agrupar todas las tareas de documentación: objetivo, stakeholders, metodología, historias de usuario, tablero, milestones y ajuste metodológico.
EP-02 AUTENTICACIÓN	Agrupar las historias y tareas relacionadas con login JWT, verificación de roles, encriptación de contraseñas y rutas protegidas.
EP-03 GESTIÓN DE BUGS	Agrupar las historias y tareas del CRUD de bugs: formulario de reporte, historial con filtros, edición y permisos por rol.
EP-04 PANEL DE ADMINISTRACIÓN	Agrupar las historias y tareas del panel de usuarios: tabla, creación, edición y eliminación de cuentas.



Control de Versiones – GitHub

El código fuente y la documentación del proyecto se gestionaron mediante **GitHub** como sistema de control de versiones, utilizando **Conventional Commits** como estándar para los mensajes del historial.

El repositorio es público y contiene el código fuente completo de **BugLog**, el archivo **parcial-2-ap-acn5bv-galarza-salazar-williams.md** con la documentación del proyecto versionada de forma incremental en el *branch docs*.

Link al Repositorio	https://github.com/MarWG/parcial-2-ap-acn5bv-galarza-salazar-williams
---------------------	---

Cada sección del informe fue incorporada en un commit separado, permitiendo que el historial de GitHub refleje la evolución del documento de forma trazable. Los commits siguen el formato tipo: *descripción (versión)*

El usuario **sergiomedinaio** fue agregado como colaborador externo del repositorio para permitir el acceso del profesor durante la evaluación.

Sprints, Milestones y Asignaciones

El proyecto fue organizado en cuatro sprints dentro de Jira, cada uno con un nombre, un objetivo definido y un conjunto de historias de usuario estimadas en Story Points. Esta estructura responde directamente al concepto de milestone: cada sprint representa una entrega incremental con funcionalidad real y verificable, que agrega valor sobre lo construido en el sprint anterior.

La decisión de dividir el trabajo en cuatro iteraciones responde a la naturaleza del proyecto. Los primeros tres sprints cubren el desarrollo técnico de BugLog en orden de dependencia: no es posible construir el CRUD de bugs sin tener primero la autenticación funcionando, ni tiene sentido implementar el panel de administración de usuarios sin que el sistema de roles esté consolidado. El cuarto sprint agrupa toda la documentación del proyecto, tratándola como un entregable propio con su propio backlog y sus propias tareas asignadas.

Sprint 1 – Autenticación y Base del Proyecto

Objetivo: establecer la base del sistema con autenticación JWT, encriptación de contraseñas y rutas protegidas por rol.

Este sprint es el fundamento sobre el que se construye todo lo demás. Sin un sistema de autenticación funcional no es posible diferenciar entre un Tester y un Administrador, ni proteger los endpoints que solo deben ser accesibles para cada rol. Las cuatro historias de este sprint cubren el login, la verificación de roles en cada request, el almacenamiento seguro de contraseñas con bcrypt y la redirección automática al login cuando la sesión no está activa.

Sprint 2 – CRUD de Bugs

Objetivo: implementar el flujo completo de reporte y gestión de bugs, incluyendo la diferenciación de permisos entre Tester y Administrador.

Este sprint representa el núcleo funcional de BugLog. Contempla el formulario de reporte con validaciones en tiempo real, el historial filtrable por gravedad, la edición de reportes propios y la posibilidad de que el Administrador intervenga sobre cualquier reporte sin restricción de propietario.

Sprint 3 – Panel de Administración

Objetivo: implementar el panel exclusivo para administradores que permite gestionar todos los usuarios registrados en el sistema.

Este sprint cierra el desarrollo técnico de BugLog incorporando el CRUD completo de usuarios, accesible únicamente para el rol Admin. Incluye la tabla de usuarios, la creación de nuevas cuentas con encriptación de contraseña, la edición de datos y rol, y la eliminación con protección de cuenta propia.

Sprint 4 – Documentación y Cierre

Objetivo: construir de forma incremental la documentación completa del proyecto, versionando cada sección como un commit independiente en GitHub.

A diferencia de los sprints anteriores, este sprint no produce código funcional sino el informe del proyecto. Cada tarea corresponde a una sección y se considera completada cuando el commit correspondiente está registrado en el historial de GitHub con el mensaje y la versión correctos.

Sprint	Nombre	Objetivo
Sprint 1	Autenticación y Base	Login JWT, roles, rutas protegidas, bcrypt
Sprint 2	CRUD de Bugs	Reporte, historial, edición, permisos por rol, portadas
Sprint 3	Panel de Administración	CRUD de usuarios exclusivo para admin
Sprint 4	Documentación y Cierre	OKR, stakeholders, metodología, etc.

Asignaciones

Las tareas fueron distribuidas entre los tres integrantes del equipo buscando equilibrio en la carga de trabajo y coherencia técnica entre historias relacionadas. Se priorizó que cada integrante tuviera participación visible en los tres sprints de desarrollo.

Integrante	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Mariano	SCRUM-5, 6, 7, 8, 26, 27, 28, 29	—	—	SCRUM-40, 41, 25
Anthony	—	SCRUM-44, 45, 11, 12, 17, 30, 31, 32, 33	—	SCRUM-38, 43
Lukas	—	—	SCRUM-13, 14, 15, 16, 34, 35, 36, 37	SCRUM-39, 42

Ajuste Metodológico

Durante la planificación inicial del proyecto los sprints fueron definidos exclusivamente en función del desarrollo técnico de BugLog: autenticación, CRUD de bugs y panel de administración. Las tareas de documentación no habían sido contempladas como ítems formales dentro del tablero Jira ni asignadas a ningún sprint, lo que generaba incertidumbre sobre cuándo y cómo iban a ser realizadas.

A medida que el equipo avanzó con los sprints de desarrollo, quedó claro que tratar la documentación como una tarea informal paralela al código presentaba un problema: no había forma de hacer seguimiento de su avance en el tablero, no existían responsables definidos para cada sección y el versionado incremental que exige la consigna no estaba garantizado.

Frente a esta situación el equipo decidió incorporar un cuarto sprint dedicado exclusivamente a la documentación, con sus propias historias, asignaciones y criterios de finalización. Cada sección del informe fue convertida en una tarjeta de Jira bajo la épica EP-01 Gestión del Proyecto, estimada en story points.

Este ajuste permitió que la documentación tuviera el mismo nivel de trazabilidad y seguimiento que el desarrollo técnico, alineando el proceso de trabajo con los principios de Scrum: nada está terminado si no está planificado, asignado y verificable en el tablero.